

8-bit ALU in SystemVerilog

Project Explanation — Presentation Guide

The Big Picture

This project is an **8-bit ALU** — an Arithmetic Logic Unit. An ALU is the heart of every CPU. It takes two numbers and an operation code, does the math or logic, and outputs a result plus three status flags. This ALU is written in **SystemVerilog**, a hardware description language — you're describing a real circuit, not writing a software program.

The project has 4 meaningful files:

File	Purpose
alu.sv	The actual ALU hardware module
alu_tb.sv	Testbench — feeds inputs, checks outputs
Makefile	Build automation (compile + run)
README.md	Documentation

File 1: alu.sv — The ALU Hardware

This is the main hardware blueprint. It describes a combinational circuit — no clock, no memory, pure wires and logic gates.

The Interface (inputs & outputs)

```
module alu (  
    input  logic [7:0] A,      // 8-bit number 1  
    input  logic [7:0] B,      // 8-bit number 2  
    input  logic [3:0] op,     // 4-bit operation selector (0-8)  
    output logic [7:0] result, // 8-bit answer  
    output logic      Z,       // Zero flag  
    output logic      N,       // Negative flag  
    output logic      V        // Overflow flag  
);
```

A and **B** are the two operands (8 bits = 0–255 unsigned, or -128 to 127 signed). **op** is a 4-bit code selecting the operation. The outputs are the result and three status flags.

Operation Codes

op code	Binary	Operation
ADD	0000	Addition

SUB	0001	Subtraction
MUL	0010	Multiplication
DIV	0011	Division
AND	0100	Bitwise AND
OR	0101	Bitwise OR
XOR	0110	Bitwise XOR
SHL	0111	Left Shift
SHR	1000	Right Shift

Full-Width Arithmetic — Why Extra Bits?

```

wire [8:0]      add_full = {1'b0, A} + {1'b0, B}; // 9 bits
wire [8:0]      sub_full = {1'b0, A} - {1'b0, B}; // 9 bits
wire signed [15:0] mul_full = A_s * B_s;          // 16 bits
wire [7:0]      div_val  = (B == 8'b0) ? 8'hFF : (A_s / B_s);

```

The result is always 8 bits, but during computation you need **more bits to detect overflow**. `add_full` is 9 bits — the extra bit reveals if addition "spilled" past 8 bits. `mul_full` is 16 bits because 8×8 can produce up to a 16-bit result. `div_val` uses a ternary: if B is zero, return 0xFF (an error sentinel) instead of crashing.

Overflow Detection

```

wire add_ovf = (A[7] == B[7]) && (add_sign != A[7]);
wire sub_ovf = (A[7] != B[7]) && (sub_sign == B[7]);
wire mul_ovf = !(mul_hi_all_zero || mul_hi_all_one);

```

The **V (Overflow) flag** means the result doesn't fit in 8 signed bits:

- **ADD overflow:** both inputs same sign, but result has different sign. E.g. $127+1 = -128$ in 8-bit — wrong, so `V=1`.
- **SUB overflow:** inputs have different signs but result has same sign as B.
- **MUL overflow:** checks if top 9 bits of 16-bit result are all zeros (positive OK) or all ones (negative OK). If neither — overflow.

The Main Logic Block — `always_comb`

```

always_comb begin
    case (op)
        ADD: begin result = add_res; V = add_ovf; end
        SUB: begin result = sub_res; V = sub_ovf; end
        MUL: begin result = mul_res; V = mul_ovf; end
        DIV: begin result = div_val; V = 1'b0; end
        AND_: result = A & B;
        OR_:  result = A | B;
        XOR_: result = A ^ B;
        SHL:  result = A << shamt;
        SHR:  result = A >> shamt;
        default: result = 8'b0;
    endcase
end

```

```
end
```

always_comb = combinational logic. This block re-runs *instantly* any time an input changes. No clock. No memory. Pure circuit: input in → output out, immediately. The case(op) works like a switch statement, selecting the operation.

The Flags

```
assign Z = (result == 8'b0); // 1 if result is zero
assign N = result[7];        // 1 if top bit is 1 (negative in signed)
```

assign = continuous assignment, also combinational. Z and N update automatically whenever result changes. These three flags (Z, N, V) are exactly what a real CPU uses after every ALU operation to make decisions — branches, conditionals, comparisons.

File 2: alu_tb.sv — The Testbench

This is **not hardware**. It's a simulation driver — code that feeds inputs into the ALU and prints the outputs to verify it works correctly.

Setup

```
`timescale 1ns/1ps
alu dut (.A(A), .B(B), .op(op), .result(result), .Z(Z), .N(N), .V(V));
```

`timescale 1ns/1ps sets the simulation time unit to 1 nanosecond with 1 picosecond precision. **dut** = "Device Under Test" — this instantiates your ALU and wires up the testbench signals to it.

Test Sequence

```
initial begin
    $dumpfile("alu.vcd");
    $dumpvars(0, alu_tb);

    A=8'd127; B=8'd1; op=4'b0000; #1;
    $display("ADD | %0d %0d | %0d | %b %b %b <- overflow", ...);
    ...
    $finish;
end
```

initial begin runs once at simulation start. **\$dumpfile/\$dumpvars** records all signal changes to alu.vcd — a waveform file you can visualize. **#1** waits 1 nanosecond for the combinational circuit to settle. **\$display** prints to the console like printf.

Test Cases Covered

Operation	Test Case	What it checks
ADD	10 + 20	Normal addition
ADD	127 + 1	Signed overflow (V flag)
SUB	30 - 10	Normal subtraction
SUB	0 - 1	Negative result (N flag)

MUL	5 * 6	Normal multiplication
MUL	50 * 10	Overflow (V flag)
DIV	100 / 4	Normal division
DIV	50 / 0	Divide by zero (returns 0xFF)
AND	0xF0 & 0x0F	Bitwise AND
OR	0xF0 0x0F	Bitwise OR
XOR	0xFF ^ 0x0F	Bitwise XOR
SHL	0x0F << 2	Left shift
SHR	0xF0 >> 2	Right shift
SUB	5 - 5	Zero flag (Z flag)

File 3: Makefile — Build Automation

```

SIM = iverilog          # the compiler (Icarus Verilog)
VVP = vvp               # the simulator runtime
SRC = alu.sv alu_tb.sv
OUT = alu.out

```

Command	What it does
make	Compile both .sv files → run simulation → print results
make wave	Run simulation → open alu.vcd waveform in VS Code
make clean	Delete alu.out and alu.vcd

The flow: **iverilog** compiles SystemVerilog into a simulation binary, then **vvp** executes it.

How It All Fits Together

```

You type: make
  |
  v
iverilog compiles alu.sv + alu_tb.sv --> alu.out
  |
  v
vvp runs alu.out (simulates the circuit)
  |
  v
Testbench sets A, B, op
  |
  v
ALU always_comb fires instantly --> result + Z, N, V appear
  |
  v
$display prints table to terminal
  |
  v
$dumpvars writes every signal change to alu.vcd (waveform file)

```

Key Points for the Presentation

- **always_comb** = **pure combinational circuit**. No clock, no memory. Output changes the instant input changes.
- **9-bit and 16-bit intermediate wires** exist specifically to detect overflow in 8-bit results — a critical design decision.
- **Three flags (Z, N, V)** are what a real CPU uses after every ALU operation to make decisions — branches, conditionals, comparisons.
- **The testbench is not synthesized into hardware** — it's a simulation-only driver that exercises all edge cases.
- **Division by zero is handled safely** by returning 0xFF instead of crashing the simulation.
- **localparam** constants give readable names to the 4-bit binary op codes — makes the code self-documenting.